

# Nikon Wireless Importer 技术方案文档

## 目录

|                                       |          |
|---------------------------------------|----------|
| <b>Nikon Wireless Importer 技术方案文档</b> | <b>3</b> |
| 1. 项目背景                               | 3        |
| 2. 产品定位                               | 3        |
| 2.1 一句话定位                             | 3        |
| 2.2 核心场景                              | 3        |
| 场景 A: 手机热点模式, 推荐主流程                   | 3        |
| 场景 B: 家庭/办公室局域网模式                     | 3        |
| 场景 C: 相机 AP 模式, 兜底流程                  | 4        |
| 3. 技术选型结论                             | 4        |
| 3.1 为什么不是纯 Flutter                    | 4        |
| 3.2 为什么核心层用 Rust                      | 4        |
| 3.3 为什么使用 UniFFI                      | 5        |
| 4. 目标平台规划                             | 5        |
| 4.1 V0.1: Android 优先                  | 5        |
| 4.2 V0.2: Android 完整体验                | 5        |
| 4.3 V0.3: iOS                         | 6        |
| 4.4 V0.4: Desktop                     | 6        |
| 5. 总体架构                               | 6        |
| 6. 代码仓库结构                             | 7        |
| 7. 核心数据模型                             | 9        |
| 7.1 CameraEndpoint                    | 9        |
| 7.2 CameraInfo                        | 9        |
| 7.3 CameraObject                      | 9        |
| 7.4 Raw + JPEG 组合模型                   | 10       |
| 8. PTP/IP 协议设计                        | 10       |
| 8.1 连接模型                              | 10       |
| 8.2 默认端口                              | 10       |
| 8.3 初始化流程                             | 10       |
| 8.4 基础命令集                             | 11       |
| 8.5 对象枚举流程                            | 11       |
| 8.6 缩略图加载流程                           | 11       |
| 8.7 原图下载流程                            | 12       |
| 9. Rust Core 模块设计                     | 12       |
| 9.1 PtpIpTransport                    | 12       |
| 9.2 PtpSession                        | 13       |
| 9.3 NikonCameraClient                 | 13       |
| 9.4 DownloadQueue                     | 13       |
| 10. Flutter UI 设计                     | 14       |
| 10.1 页面结构                             | 14       |
| 10.2 ConnectScreen                    | 14       |
| 10.3 GalleryScreen                    | 14       |
| 10.4 PreviewScreen                    | 15       |
| 10.5 DownloadScreen                   | 15       |
| 11. 平台适配设计                            | 15       |

|        |                                    |    |
|--------|------------------------------------|----|
| 11.1   | Android                            | 15 |
| 11.1.1 | 网络连接                               | 15 |
| 11.1.2 | 局域网扫描                              | 16 |
| 11.1.3 | 文件保存                               | 16 |
| 11.1.4 | 后台下载                               | 16 |
| 11.2   | iOS                                | 16 |
| 11.2.1 | Local Network 权限                   | 16 |
| 11.2.2 | 网络连接                               | 17 |
| 11.2.3 | 文件保存                               | 17 |
| 11.2.4 | 后台限制                               | 17 |
| 11.3   | Desktop                            | 17 |
| 11.3.1 | 网络                                 | 17 |
| 11.3.2 | 文件保存                               | 17 |
| 11.3.3 | CLI                                | 17 |
| 12.    | 兼容性策略                              | 18 |
| 12.1   | Capability Probe                   | 18 |
| 12.2   | 机型兼容表                              | 18 |
| 13.    | 错误处理设计                             | 18 |
| 13.1   | 错误分类                               | 18 |
| 13.2   | 面向用户的错误文案                          | 19 |
|        | CameraNotFound                     | 19 |
|        | ConnectionTimeout                  | 19 |
|        | LocalNetworkPermissionDenied       | 19 |
|        | ThumbnailUnavailable               | 19 |
|        | DownloadInterrupted                | 19 |
| 14.    | 性能设计                               | 19 |
| 14.1   | 首屏目标                               | 19 |
| 14.2   | 列表加载策略                             | 19 |
| 14.3   | 缩略图策略                              | 20 |
| 14.4   | 下载策略                               | 20 |
| 15.    | 安全与隐私                              | 20 |
| 15.1   | 数据不上传云端                            | 20 |
| 15.2   | 不采集照片内容                            | 20 |
| 15.3   | 局域网访问说明                            | 20 |
| 16.    | 测试方案                               | 20 |
| 16.1   | Rust Core 单元测试                     | 20 |
| 16.2   | Mock Camera                        | 21 |
| 16.3   | 真机测试                               | 21 |
| 16.4   | 压力测试                               | 21 |
| 17.    | 里程碑计划                              | 21 |
|        | Milestone 0: 协议验证 CLI              | 21 |
|        | Milestone 1: Android 手动连接 Demo     | 22 |
|        | Milestone 2: 照片列表与缩略图              | 22 |
|        | Milestone 3: 下载闭环                  | 22 |
|        | Milestone 4: 跨端扩展                  | 22 |
| 18.    | 风险与应对                              | 23 |
|        | 风险 1: 部分 Nikon 机型不开放 PTP/IP        | 23 |
|        | 风险 2: 相机需要 SnapBridge BLE 唤醒 Wi-Fi | 23 |
|        | 风险 3: 大文件下载中断                      | 23 |
|        | 风险 4: iOS 后台能力不足                   | 23 |
|        | 风险 5: 局域网扫描慢                       | 23 |
| 19.    | 非目标范围                              | 23 |
| 20.    | 推荐开发顺序                             | 24 |
| 21.    | V0.1 验收标准                          | 24 |
| 22.    | 总结                                 | 24 |

# Nikon Wireless Importer 技术方案文档

## 1. 项目背景

尼康官方 SnapBridge / NX 系列工具在“快速预览相机内存卡照片并选择性下载”这个场景下体验不稳定，尤其是在以下需求上不够顺手：

- 不希望使用 USB 线、读卡器或其他硬件连接。
- 希望手机在导图时仍然保持互联网连接。
- 希望先快速浏览缩略图，再决定是否下载原图。
- 希望支持 JPEG、NEF、MOV/MP4 等文件。
- 希望后续扩展到 Android、iOS、桌面端、CLI 或 NAS 自动导片。

本项目目标是开发一个跨端的 Nikon 无线照片预览与下载工具。它不是完整替代 SnapBridge 的所有功能，而是聚焦于一个更明确、更高频的能力：

手机/电脑通过 Wi-Fi 局域网连接 Nikon 相机，快速枚举相机存储中的照片，加载缩略图，批量选择并下载原图。

## 2. 产品定位

### 2.1 一句话定位

一个面向 Nikon 相机的无线读卡器。

### 2.2 核心场景

#### 场景 A：手机热点模式，推荐主流程

手机开启热点，手机继续使用 4G/5G 上网  
↓  
Nikon 相机连接手机热点  
↓  
App 在热点局域网内发现相机  
↓  
App 通过 PTP/IP 读取照片、缩略图、原图

优点：

- 手机不会因为连接相机 Wi-Fi 而断网。
- 适合户外、旅拍、活动现场。
- 用户只需要让相机接入手机热点。

缺点：

- 需要相机支持 STA / Infrastructure / Connect to PC / 连接到已有网络。
- 需要发现相机被分配到的局域网 IP。

#### 场景 B：家庭/办公室同局域网模式

手机/电脑连接家庭 Wi-Fi  
相机连接同一个 Wi-Fi  
App 自动扫描并连接相机

优点：

- 体验最稳定。
- 适合批量导图、回家后整理照片。

- 可扩展桌面端、NAS 自动导片。

缺点：

- 外出时依赖可用 Wi-Fi。
- 相机端配置路径因机型而异。

### 场景 C：相机 AP 模式，兜底流程

相机开启自己的 Wi-Fi 热点  
手机连接相机 Wi-Fi  
App 默认访问 192.168.1.1:15740

优点：

- 不依赖路由器。
- 对部分相机更容易启动。
- 适合作为兼容兜底。

缺点：

- 手机 Wi-Fi 被相机占用，通常会断开互联网。
- 某些 Android 设备会提示“此 Wi-Fi 无互联网”。

## 3. 技术选型结论

最终方案：

```
Flutter UI
├── Android 原生插件
├── iOS 原生插件
├── Desktop 原生适配
└── Rust Core via UniFFI / FFI
    ├── PTP/IP 协议层
    ├── PTP 命令层
    ├── Nikon Camera Client
    ├── 对象索引与缩略图调度
    ├── 下载队列
    └── 错误恢复与日志
```

### 3.1 为什么不是纯 Flutter

Flutter 适合做 UI 和跨端业务流，但本项目有大量平台相关能力：

- Android 指定网络 Socket。
- Android MediaStore 保存照片。
- Android Foreground Service 下载队列。
- iOS Local Network 权限。
- iOS Photos / Files 保存。
- iOS 后台任务限制。
- 桌面端文件系统与局域网扫描。

因此不能把所有能力都写在 Dart 层。

### 3.2 为什么核心层用 Rust

PTP/IP 是二进制协议，涉及：

- TCP 长连接。
- 二进制包解析。
- 小端/大端处理。
- 事务 ID 管理。
- 大文件流式传输。
- 连接状态机。
- 断线重试。
- 跨端复用。

Rust 适合实现这类稳定、可复用、低层协议核心。后续同一个 Core 可以复用到：

- Android App。
- iOS App。
- macOS / Windows / Linux 桌面端。
- CLI 工具。
- NAS 自动导片服务。

### 3.3 为什么使用 UniFFI

UniFFI 负责从 Rust 生成 Kotlin / Swift 等语言绑定，减少手写 FFI 胶水代码。

第一阶段可采用：

```
Flutter Dart
  ↓ Platform Channel
Android Kotlin / iOS Swift
  ↓ UniFFI generated binding
Rust Core
```

后续若性能敏感或希望减少中间层，可评估：

```
Flutter Dart
  ↓ dart:ffi
Rust C ABI wrapper
```

但 V0.1 不建议直接从 Dart FFI 开始，因为 Android/iOS 本来就需要大量原生平台能力，Platform Channel 更利于统一封装。

## 4. 目标平台规划

### 4.1 V0.1: Android 优先

目标：验证核心闭环。

功能：

- 手动输入 IP。
- 扫描局域网内开放 15740 端口的设备。
- 建立 PTP/IP 连接。
- 获取相机信息。
- 读取存储列表。
- 枚举照片对象。
- 读取缩略图。
- 下载单张 JPEG / NEF。

### 4.2 V0.2: Android 完整体验

功能：

- 手机热点模式优化。
- 自动发现相机。
- RAW+JPEG 合并展示。
- 批量下载。
- 下载进度。
- 前台服务保活。
- MediaStore 保存。
- 本地索引缓存。
- 断线重试。

#### 4.3 V0.3: iOS

功能：

- Local Network 权限处理。
- 手动输入 IP。
- 局域网扫描。
- PTP/IP 连接。
- 照片列表与缩略图。
- 下载到 Files / Photos。

限制：

- iOS 后台下载限制更严格。
- 局域网扫描与权限提示需要精心处理。
- 不优先做长时间后台批量下载。

#### 4.4 V0.4: Desktop

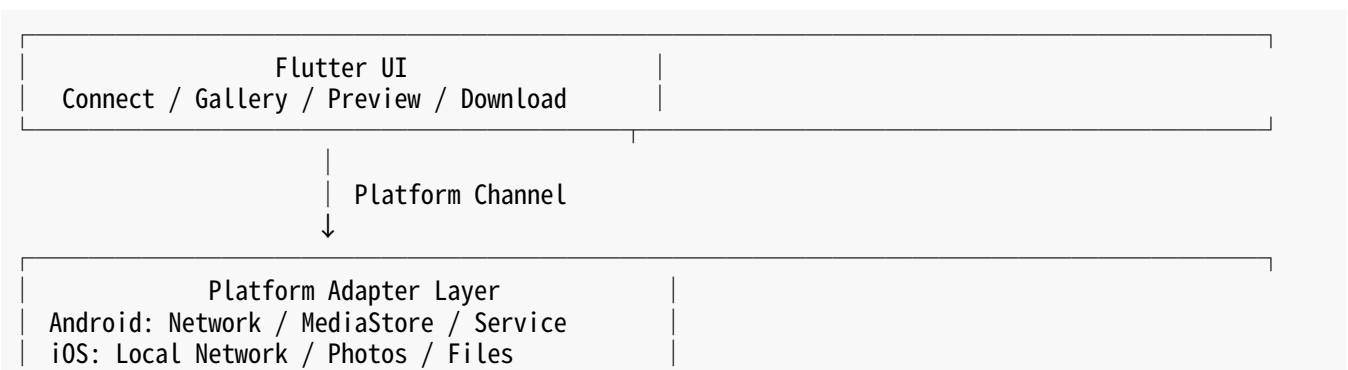
目标平台：

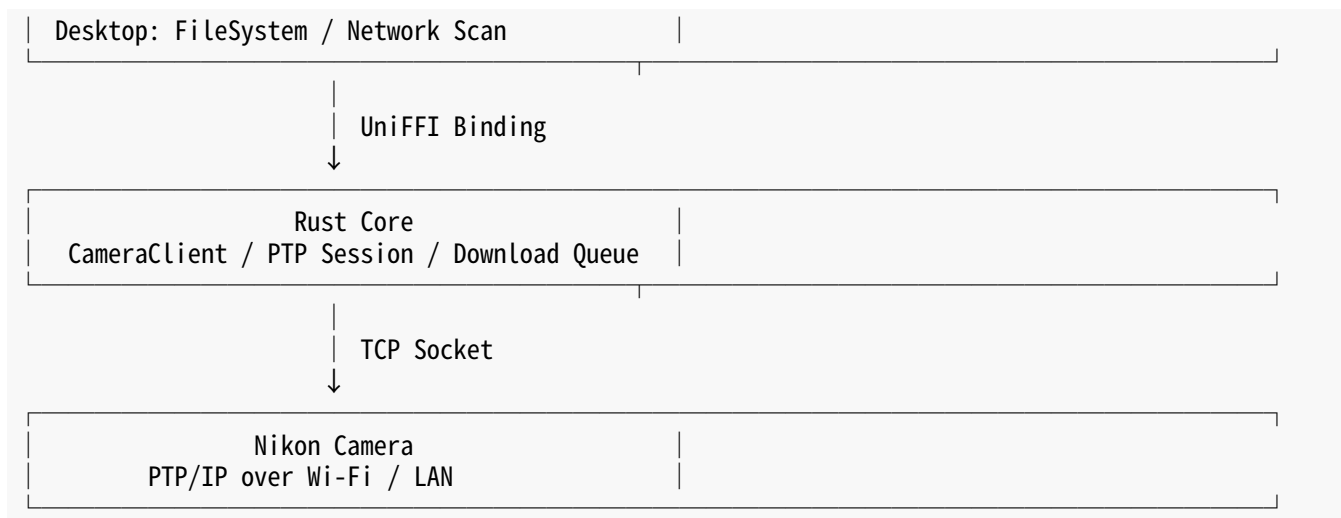
- macOS。
- Windows。
- Linux。

功能：

- 桌面批量导片。
- 指定目录保存。
- 按日期自动建目录。
- 增量导入。
- CLI 工具。
- NAS 自动导片模式。

### 5. 总体架构





## 6. 代码仓库结构

nikon-wireless-importer/

```
core/  
Cargo.toml  
src/  
lib.rs  
error.rs  
model/  
mod.rs  
endpoint.rs  
camera_info.rs  
camera_object.rs  
object_format.rs  
ptp_ip/  
mod.rs  
packet.rs  
transport.rs  
init.rs  
event.rs  
ptp/  
mod.rs  
operation.rs  
response.rs  
dataset.rs  
session.rs  
camera/  
mod.rs  
client.rs  
object_repository.rs  
capability.rs  
download/  
mod.rs  
queue.rs  
progress.rs
```

```

    sink.rs
  scanner/
    mod.rs
    port_scan.rs
  log/
    mod.rs
  uniffi.toml
  src/nikon_importer.udl

apps/
  mobile_flutter/
    pubspec.yaml
    lib/
      main.dart
      app.dart
      screens/
        connect_screen.dart
        gallery_screen.dart
        preview_screen.dart
        download_screen.dart
        settings_screen.dart
      view_models/
        connect_view_model.dart
        gallery_view_model.dart
        download_view_model.dart
      models/
        camera_endpoint.dart
        camera_object_view.dart
      platform/
        nikon_platform_api.dart
      android/
        app/src/main/kotlin/.../
          NikonPlatformPlugin.kt
          AndroidNetworkAdapter.kt
          AndroidMediaStoreSaver.kt
          AndroidDownloadService.kt
      ios/
        Runner/
          NikonPlatformPlugin.swift
          LocalNetworkPermission.swift
          IOSPhotoSaver.swift

  desktop_flutter/
    lib/
      macos/
      windows/
      linux/

tools/
  cli/
    Cargo.toml
    src/main.rs

docs/
  protocol.md

```



## 7. 核心数据模型

### 7.1 CameraEndpoint

```
pub struct CameraEndpoint {  
    pub host: String,  
    pub port: u16,  
    pub source: EndpointSource,  
}  
  
pub enum EndpointSource {  
    Manual,  
    LanScan,  
    CameraApDefault,  
    PreviousSuccessful,  
}
```

### 7.2 CameraInfo

```
pub struct CameraInfo {  
    pub manufacturer: String,  
    pub model: String,  
    pub serial_number: Option<String>,  
    pub firmware_version: Option<String>,  
    pub supported_operations: Vec<u16>,  
    pub supported_formats: Vec<u16>,  
}
```

### 7.3 CameraObject

```
pub struct CameraObject {  
    pub handle: u32,  
    pub storage_id: u32,  
    pub filename: String,  
    pub size_bytes: u64,  
    pub format: ObjectFormat,  
    pub capture_time_ms: Option<i64>,  
    pub modified_time_ms: Option<i64>,  
    pub width: Option<u32>,  
    pub height: Option<u32>,  
    pub thumb_available: bool,  
    pub downloaded: bool,  
    pub group_key: Option<String>,  
}  
  
pub enum ObjectFormat {  
    Jpeg,  
    Nef,  
    Mov,
```

```

    Mp4,
    Tiff,
    Unknown,
}

```

## 7.4 Raw + JPEG 组合模型

```

pub struct CameraAssetGroup {
    pub group_key: String,
    pub primary: CameraObject,
    pub jpeg: Option<CameraObject>,
    pub raw: Option<CameraObject>,
    pub video: Option<CameraObject>,
}

```

组合规则：

```

DSC_1234.JPG
DSC_1234.NEF
=> group_key = DSC_1234
=> UI 显示为一个卡片，角标 RAW+JPG

```

## 8. PTP/IP 协议设计

### 8.1 连接模型

PTP/IP 使用两个 TCP 连接：

```

Command/Data Connection
Event Connection

```

Command/Data Connection 用于发送命令和接收数据。Event Connection 用于接收相机事件。

### 8.2 默认端口

默认端口：

15740

### 8.3 初始化流程

```

TCP connect camera_host:15740
↓
Send InitCommandRequest
↓
Receive InitCommandAck
↓
Open second TCP connection
↓
Send InitEventRequest
↓
Receive InitEventAck
↓
OpenSession

```

↓  
GetDeviceInfo

## 8.4 基础命令集

V0.1 只实现必要命令：

```
pub enum PtpOperationCode {  
    GetDeviceInfo = 0x1001,  
    OpenSession = 0x1002,  
    CloseSession = 0x1003,  
    GetStorageIDs = 0x1004,  
    GetStorageInfo = 0x1005,  
    GetObjectHandles = 0x1007,  
    GetObjectInfo = 0x1008,  
    GetObject = 0x1009,  
    GetThumb = 0x100A,  
}
```

## 8.5 对象枚举流程

```
OpenSession  
↓  
GetStorageIDs  
↓  
for each storage_id:  
    GetStorageInfo  
    GetObjectHandles(storage_id)  
↓  
for each object_handle:  
    GetObjectInfo(handle)  
↓  
filter supported object types  
↓  
sort by capture_time desc
```

## 8.6 缩略图加载流程

```
用户进入 GalleryScreen  
↓  
优先显示对象列表占位  
↓  
按屏幕可见范围请求 GetThumb  
↓  
Rust Core 返回 bytes  
↓  
Platform / Flutter 写入缩略图缓存  
↓  
UI 渐进显示
```

原则：

- 不一次性拉全量缩略图。
- 优先加载首屏和用户滑动方向附近图片。
- 失败的缩略图允许重试。

- 若 GetThumb 不支持，则 UI 显示文件占位图。

## 8.7 原图下载流程

用户选择文件  
↓  
创建 DownloadTask  
↓  
GetObject(handle)  
↓  
流式写入临时文件  
↓  
校验大小  
↓  
移动到目标目录 / MediaStore / Photos  
↓  
标记 downloaded

临时文件策略：

filename.tmp  
下载完成后 rename 为 filename

失败恢复策略：

- V0.1: 失败后重下。
- V0.2: 支持任务级重试。
- V0.3: 评估分块恢复能力；如果相机不支持可靠 Range/Partial Object，则不做断点续传，只做重试。

---

## 9. Rust Core 模块设计

### 9.1 PtpIpTransport

职责：

- 管理 TCP socket。
- 发送 PTP/IP packet。
- 接收 PTP/IP packet。
- 维护 command connection。
- 维护 event connection。
- 提供超时控制。

接口草案：

```
pub struct PtpIpTransport {}

impl PtpIpTransport {
    pub async fn connect(endpoint: CameraEndpoint) -> Result<Self>;
    pub async fn init_command(&mut self) -> Result<()>;
    pub async fn init_event(&mut self) -> Result<()>;
    pub async fn send_packet(&mut self, packet: PtpIpPacket) -> Result<()>;
    pub async fn read_packet(&mut self) -> Result<PtpIpPacket>;
    pub async fn close(&mut self) -> Result<()>;
}
```

## 9.2 PtpSession

职责：

- 维护 session\_id。
- 维护 transaction\_id。
- 封装 PTP operation。
- 解析 response code。

接口草案：

```
pub struct PtpSession {
    session_id: u32,
    transaction_id: u32,
}

impl PtpSession {
    pub async fn open(&mut self) -> Result<()>;
    pub async fn close(&mut self) -> Result<()>;
    pub async fn operation(
        &mut self,
        code: u16,
        params: Vec<u32>,
        data: Option<Vec<u8>>,
    ) -> Result<PtpOperationResult>;
}
```

## 9.3 NikonCameraClient

职责：

- 面向业务封装相机操作。
- 屏蔽 PTP 细节。
- 提供稳定 API 给 UniFFI。

接口草案：

```
pub struct NikonCameraClient {}

impl NikonCameraClient {
    pub async fn connect(endpoint: CameraEndpoint) -> Result<Self>;
    pub async fn get_camera_info(&mut self) -> Result<CameraInfo>;
    pub async fn list_objects(&mut self) -> Result<Vec<CameraObject>>;
    pub async fn get_thumbnail(&mut self, handle: u32) -> Result<Vec<u8>>;
    pub async fn download_object(
        &mut self,
        handle: u32,
        sink: DownloadSink,
    ) -> Result<()>;
    pub async fn close(&mut self) -> Result<()>;
}
```

## 9.4 DownloadQueue

职责：

- 管理下载任务。
- 控制并发。
- 记录进度。

- 失败重试。
- 取消任务。

并发策略：

默认下载并发：1

缩略图并发：2~4

原因：

- 相机端连接能力有限。
- 大文件并发下载容易导致相机断开。
- 缩略图可以适度并发，但也要限流。

---

## 10. Flutter UI 设计

### 10.1 页面结构

```
ConnectScreen
↓
GalleryScreen
↓
PreviewScreen
↓
DownloadScreen
↓
SettingsScreen
```

### 10.2 ConnectScreen

核心组件：

连接方式选择：

[自动扫描局域网]

[手动输入 IP]

[相机热点模式]

状态展示：

未连接 / 扫描中 / 已发现 / 连接中 / 已连接 / 失败

连接方式说明：

推荐：手机开热点，让相机连接手机热点。

如果你的相机不支持连接已有 Wi-Fi，可以使用相机热点模式。

### 10.3 GalleryScreen

核心组件：

顶部筛选：全部 / JPG / RAW / 视频 / 未下载

排序：拍摄时间倒序 / 文件名

视图：瀑布流 / 网格

底部操作栏：已选 N 张 / 下载原图

卡片展示：

缩略图  
文件名  
RAW+JPG 角标  
文件大小  
已下载标记

## 10.4 PreviewScreen

功能：

- 查看更大预览图。
- 显示文件信息。
- 下载 JPEG。
- 下载 RAW。
- 下载整组 RAW+JPEG。

## 10.5 DownloadScreen

展示：

正在下载  
等待中  
已完成  
失败

支持：

- 暂停。
- 取消。
- 重试。
- 打开保存位置。

---

## 11. 平台适配设计

### 11.1 Android

#### 11.1.1 网络连接

目标：

访问相机：走热点/局域网 Wi-Fi  
访问互联网：仍然允许走蜂窝数据

实现策略：

- 使用 ConnectivityManager 获取当前可用网络。
- 对相机连接使用特定 Network 的 socketFactory。
- 避免全局 bindProcessToNetwork，防止整个 App 网络被绑到无互联网 Wi-Fi。

Android Platform Adapter：

```
class AndroidNetworkAdapter {  
    fun getAvailableNetworks(): List<AndroidNetworkInfo>  
    fun createSocketOnNetwork(networkId: String, host: String, port: Int): Socket  
    fun getLikelySubnets(): List<String>  
}
```

### 11.1.2 局域网扫描

策略：

- 优先从当前网络推断网段。
- 端口扫描 15740。
- 超时时间 200~500ms。
- 并发限制 32~64。
- 扫描结果按响应速度排序。

兜底网段：

```
192.168.1.0/24
192.168.0.0/24
192.168.43.0/24
192.168.231.0/24
172.20.10.0/28
```

### 11.1.3 文件保存

JPEG：

```
Pictures/NikonImporter/YYYY-MM-DD/
```

NEF：

```
Pictures/NikonImporter/RAW/YYYY-MM-DD/
```

视频：

```
Movies/NikonImporter/YYYY-MM-DD/
```

保存方式：

- Android 10+ 使用 MediaStore。
- 低版本按权限策略降级。
- 下载期间先写临时文件，完成后 publish。

### 11.1.4 后台下载

Android 端使用 Foreground Service 显示持续下载通知。

通知内容：

```
正在下载 3/28
当前文件 DSC_1234.NEF
进度 42%
```

V0.1 可以先要求 App 前台运行。V0.2 再引入 Foreground Service。

---

## 11.2 iOS

### 11.2.1 Local Network 权限

iOS 访问局域网设备需要处理本地网络隐私权限。

Info.plist 需要配置：

```
NSLocalNetworkUsageDescription
```

如果后续使用 Bonjour/mDNS，还需要配置：



## NSBonjourServices

V0.3 初期不依赖 Bonjour，先使用手动 IP + 端口扫描。

### 11.2.2 网络连接

实现方式：

- Rust Core 可直接使用 TCP。
- iOS 原生层负责权限触发与网络状态检查。
- 如需更细控制，可用 Network.framework 的 TCP 能力。

### 11.2.3 文件保存

保存策略：

- JPEG 可保存到 Photos。
- NEF 优先保存到 Files / App Documents。
- 用户可从 App 内导出到其他位置。

### 11.2.4 后台限制

iOS 对后台长时间下载和局域网连接限制较多，V0.3 不承诺长期后台下载。

推荐设计：

App 前台批量下载  
屏幕常亮提示  
失败任务可恢复重试

---

## 11.3 Desktop

### 11.3.1 网络

桌面端可直接使用 Rust TCP。

功能：

- 扫描局域网。
- 手动输入 IP。
- 保存历史相机地址。

### 11.3.2 文件保存

默认目录：

~/Pictures/NikonImporter/YYYY-MM-DD/

支持用户自定义目录。

### 11.3.3 CLI

命令设计：

```
nikon-importer scan
nikon-importer info --host 192.168.43.21
nikon-importer list --host 192.168.43.21
nikon-importer thumb --host 192.168.43.21 --handle 123
```

```
nikon-importer pull --host 192.168.43.21 --latest 20 --output ./photos
nikon-importer sync --host 192.168.43.21 --output ./photos --skip-existing
```

CLI 是验证 Rust Core 最好的工具，建议与 Android V0.1 并行开发。

---

## 12. 兼容性策略

### 12.1 Capability Probe

连接后立即执行：

GetDeviceInfo



读取 manufacturer / model / supported\_operations / supported\_formats



生成 CameraCapability

CameraCapability:

```
pub struct CameraCapability {
    pub supports_get_thumb: bool,
    pub supports_get_object: bool,
    pub supports_storage_info: bool,
    pub supports_raw_download: bool,
    pub supports_video_download: bool,
}
```

### 12.2 机型兼容表

维护 compatibility.md:

| Model | Mode          | Status  | Notes     |
|-------|---------------|---------|-----------|
| Zf    | Phone hotspot | Unknown | Need test |
| Z6II  | Camera AP     | Unknown | Need test |
| Z5    | LAN           | Unknown | Need test |
| D780  | LAN           | Unknown | Need test |

每次测试记录：

- 相机型号。
  - 固件版本。
  - 连接模式。
  - 是否能发现 15740。
  - 是否能 InitCommand。
  - 是否能 GetDeviceInfo。
  - 是否能 GetThumb。
  - 是否能 GetObject。
  - 大文件下载稳定性。
- 

## 13. 错误处理设计

### 13.1 错误分类

```
pub enum ImporterError {
    NetworkUnavailable,
```

```
CameraNotFound,  
ConnectionTimeout,  
PtpInitFailed,  
SessionOpenFailed,  
UnsupportedOperation,  
ObjectNotFound,  
ThumbnailUnavailable,  
DownloadInterrupted,  
StoragePermissionDenied,  
LocalNetworkPermissionDenied,  
UnknownCameraResponse,  
InternalError,  
}
```

## 13.2 面向用户的错误文案

### CameraNotFound

没有发现相机。请确认相机已经连接到手机热点或同一个 Wi-Fi，然后重新扫描。

### ConnectionTimeout

连接超时。相机可能已经休眠，或当前网络无法访问相机。

### LocalNetworkPermissionDenied

需要允许访问本地网络，App 才能发现和连接相机。

### ThumbnailUnavailable

这张照片无法读取缩略图，但仍然可以尝试下载原文件。

### DownloadInterrupted

下载中断。请保持相机开启并靠近手机，然后重试。

---

## 14. 性能设计

### 14.1 首屏目标

目标体验：

连接成功后 3 秒内显示文件列表。

10 秒内逐步显示首屏缩略图。

### 14.2 列表加载策略

- 先拉对象元数据，不拉原图。
- 按拍摄时间倒序显示。
- 首次最多加载最近 300 个对象。

- 支持继续加载更多。

### 14.3 缩略图策略

- 只加载可见区域和预加载区域。
- Disk cache 以 handle + filename + size 作为 key。
- 缩略图失败不阻塞列表。
- 滚动时取消远离视口的缩略图请求。

### 14.4 下载策略

- 原图下载并发默认 1。
  - 缩略图并发默认 2~4。
  - 大文件写入使用流式写入。
  - 下载完成后校验大小。
- 

## 15. 安全与隐私

### 15.1 数据不上传云端

默认所有照片只在本地传输和保存，不上传服务器。

### 15.2 不采集照片内容

日志中禁止记录：

- 图片内容。
- 完整 EXIF 隐私信息。
- 精确 GPS 坐标。

允许记录：

- 相机型号。
- 固件版本。
- 连接模式。
- 错误码。
- 文件格式统计。
- 下载耗时。

### 15.3 局域网访问说明

首次启动时说明：

本 App 需要访问局域网设备，用于发现和连接你的相机。照片只会在你的设备和相机之间传输。

---

## 16. 测试方案

### 16.1 Rust Core 单元测试

测试范围：

- PTP/IP packet 编解码。
- PTP response 解析。
- ObjectInfo dataset 解析。
- 文件格式识别。

- RAW+JPEG groupKey。
- 错误码映射。

## 16.2 Mock Camera

实现一个本地 mock server:

```
localhost:15740
模拟 InitCommandAck
模拟 GetDeviceInfo
模拟 GetStorageIDs
模拟 GetObjectHandles
模拟 GetObjectInfo
模拟 GetThumb
模拟 GetObject
```

用途:

- CI 自动测试。
- 不依赖真实相机开发 UI。
- 复现异常响应。

## 16.3 真机测试

真机测试矩阵:

```
Android 手机 + 手机热点模式
Android 手机 + 家庭 Wi-Fi 模式
Android 手机 + 相机 AP 模式
iPhone + 家庭 Wi-Fi 模式
iPhone + 手机热点模式
Desktop + 家庭 Wi-Fi 模式
```

## 16.4 压力测试

测试项:

- 1000 张照片枚举。
- 100 张缩略图连续加载。
- 50 张 JPEG 批量下载。
- 10 张 NEF 批量下载。
- 下载中相机休眠。
- 下载中 Wi-Fi 断开。
- 下载中 App 退到后台。

---

## 17. 里程碑计划

### Milestone 0: 协议验证 CLI

目标: 先证明相机可被 PTP/IP 访问。

交付物:

```
nikon-importer-cli scan
nikon-importer-cli info --host <ip>
nikon-importer-cli list --host <ip>
```

验收:

- 能连接真实 Nikon 相机。
- 能输出型号。
- 能列出至少部分照片文件名。

### **Milestone 1: Android 手动连接 Demo**

交付物：

- Flutter Android App。
- 手动输入 IP。
- 连接相机。
- 显示相机型号。

验收：

- 手机热点/同局域网下可以连接相机。
- 连接失败有明确错误提示。

### **Milestone 2: 照片列表与缩略图**

交付物：

- GalleryScreen。
- 对象列表。
- 缩略图懒加载。
- JPG/RAW/视频筛选。

验收：

- 最近 100 张照片可浏览。
- 首屏缩略图逐步显示。

### **Milestone 3: 下载闭环**

交付物：

- 单张下载。
- 批量下载。
- MediaStore 保存。
- 下载进度。

验收：

- JPEG 可在系统相册看到。
- NEF 可在指定目录看到。
- 下载失败可重试。

### **Milestone 4: 跨端扩展**

交付物：

- iOS 手动 IP 连接。
- iOS 列表与缩略图。
- Desktop CLI。

验收：

- Rust Core 不改或少改即可复用。
- iOS 可完成基本下载。
- Desktop CLI 可批量拉取。

## 18. 风险与应对

### 风险 1：部分 Nikon 机型不开放 PTP/IP

应对：

- 维护兼容性表。
- 先支持明确可连通的机型。
- 相机 AP 模式作为兜底。
- 不承诺所有 Nikon 型号可用。

### 风险 2：相机需要 SnapBridge BLE 唤醒 Wi-Fi

应对：

- V0.1 不做 BLE 唤醒。
- 要求用户手动在相机菜单开启 Wi-Fi 或连接至电脑。
- 后续再研究 BLE 流程。

### 风险 3：大文件下载中断

应对：

- 下载并发限制为 1。
- 增加 keep-alive / 超时配置。
- 下载失败自动重试。
- 临时文件机制避免脏文件。

### 风险 4：iOS 后台能力不足

应对：

- iOS 首版只承诺前台下载。
- 下载页面提示保持 App 前台。
- 后续根据系统能力评估 Background Task。

### 风险 5：局域网扫描慢

应对：

- 优先使用上次成功 IP。
  - 优先扫描当前网段。
  - 用户可手动输入 IP。
  - 扫描并发限制。
  - 扫描中逐步展示结果。
- 

## 19. 非目标范围

V0.1 不做：

- 相机遥控拍摄。
- 实时取景 Live View。
- 修改相机参数。
- BLE 自动唤醒。
- 云同步。
- 图片编辑。
- RAW 解码编辑。
- 完整 EXIF 管理。

- 所有 Nikon 型号兼容承诺。

这些功能可以在核心导片体验稳定后再评估。

---

## 20. 推荐开发顺序

最推荐顺序：

1. Rust CLI: connect + GetDeviceInfo
2. Rust CLI: list objects
3. Rust CLI: download one object
4. Flutter Android: 手动 IP 连接
5. Flutter Android: 照片列表
6. Flutter Android: 缩略图
7. Flutter Android: 下载保存
8. Android: 局域网扫描 + 手机热点优化
9. iOS: 手动 IP + Local Network 权限
10. Desktop: CLI / GUI 扩展

关键原则：

先证明协议闭环，再做漂亮 UI。

---

## 21. V0.1 验收标准

给定一台支持 PTP/IP 的 Nikon 相机：

1. 手机开热点。
2. 相机连接手机热点。
3. App 扫描并发现相机。
4. App 显示相机型号。
5. App 显示最近照片列表。
6. App 加载首屏缩略图。
7. 用户选择一张 JPEG 并下载。
8. 用户选择一张 NEF 并下载。
9. 文件保存到手机本地目录。
10. 下载失败时可重试。

如果 V0.1 做到以上 10 点，就已经具备替代 SnapBridge 核心导片体验的基础。

---

## 22. 总结

本项目的最优路线不是“跨端 UI 框架写一切”，而是：

Rust Core 负责稳定协议能力

Flutter 负责跨端 UI

Android/iOS/Desktop 原生层负责系统能力

产品上聚焦：

快速发现相机

快速预览缩略图



选择性下载原图  
手机不断网

工程上聚焦：

协议核心独立  
平台能力隔离  
先 CLI 验证  
再移动端体验  
最后跨端扩展

这个方向能最大化复用核心能力，也能避免被某一个移动端框架或 Nikon 官方 App 体验限制住。